



Retrieval-Augmented Generation

Ground LLMs on Your Own Data

SPRING 2026

Recap - Where We Left Off

Last session, you learned to run models locally and via APIs, prompt them effectively, and use LangChain to orchestrate calls.

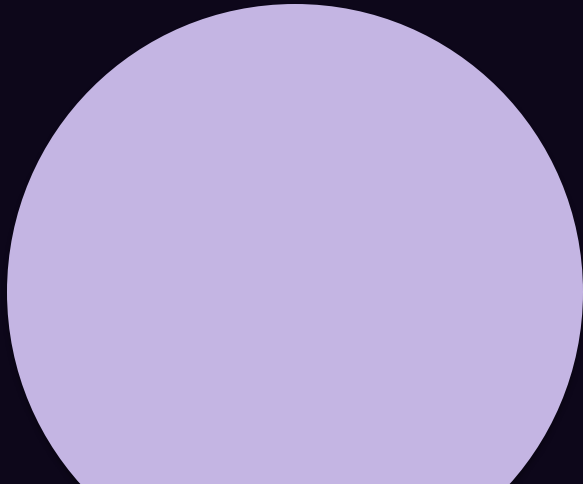
You tried to built chatbots that could hold a conversation. But ask your chatbot about your company's internal docs, yesterday's meeting notes, or a specific PDF, and it fails. It either hallucinates an answer or says "I don't have that information." RAG is the solution.

Today's Roadmap

- Why RAG? The Problem It Solves
- RAG Architecture End-to-End
- Deep Dive: Chunking Strategies
- Deep Dive: Embeddings & Embedding Models
- Deep Dive: Vector Databases
- Retrieval & Generation
- Simple vs. Better RAG
- Basic Evaluation
- Lab: Build a Full RAG Pipeline

Part 1: Why RAG?

The two fundamental problems with vanilla LLMs, and why retrieval is the answer.



Why LLMs Alone Aren't Enough

- **Knowledge cutoff:** Models only know what was in their training data. GPT-4o's knowledge stops at a certain date. Your internal docs, recent events, and proprietary data are also invisible to the model.
- **Hallucination:** When an LLM doesn't know something, it doesn't say "I don't know." It confidently generates plausible-sounding but incorrect information. This is dangerous in production.
- **No source attribution:** Even when an LLM is correct, it can't tell you where the information came from. No citations, no traceability, no trust.
- **The cost of fine-tuning:** You could fine-tune a model on your data, but that's expensive, slow, and needs to be repeated every time data changes. RAG is dynamic, you just add a document, and it's immediately searchable.

The Core Idea Behind RAG

Retrieval-Augmented Generation is simple in concept: before the LLM generates an answer, first retrieve relevant information from an external knowledge base, then pass that information alongside the user's question to the LLM. The model generates an answer grounded in the retrieved context, not just its training data.

Think of it this way: instead of asking someone to answer from memory, you hand them the relevant pages from a textbook first, then ask the question. They'll give a much better, more accurate answer.

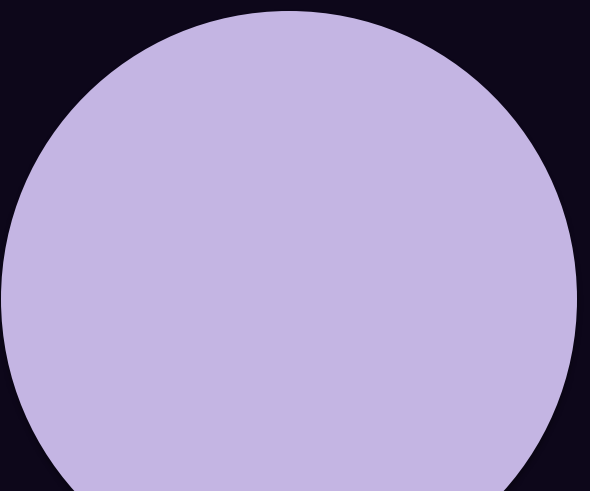
The original RAG paper was published by Meta (Lewis et al., 2020), but the practical pattern has exploded since 2023 with the rise of vector databases and LangChain.

RAG vs. Fine-Tuning vs. Long Context

- **Fine-tuning:** Update model weights on your data. Expensive (\$100s–\$1000s), takes hours/days, needs retraining when data changes, can cause catastrophic forgetting. Best for changing model behavior/style, not for adding knowledge.
- **Long context windows:** Some models accept 100K–1M tokens. Just stuff all your docs in the prompt? Works for small datasets, but: costs scale linearly with context length, retrieval accuracy degrades with very long contexts, and you'll hit limits eventually.
- **RAG:** Keep your data external. Retrieve only what's relevant (a few chunks, not everything). Cost-effective, data stays fresh, scales to millions of documents, provides source attribution. The standard approach for knowledge-grounded applications.
- **In practice:** These aren't mutually exclusive. You might fine-tune for tone/style + RAG for knowledge + long context for complex reasoning over retrieved chunks.

Part 2: RAG Architecture End-to-End

The six-step pipeline: Ingest → Chunk → Embed → Store → Retrieve → Generate



The Full RAG Pipeline

Every RAG system follows the same fundamental architecture with two phases:

Indexing Phase (offline, done once per document):

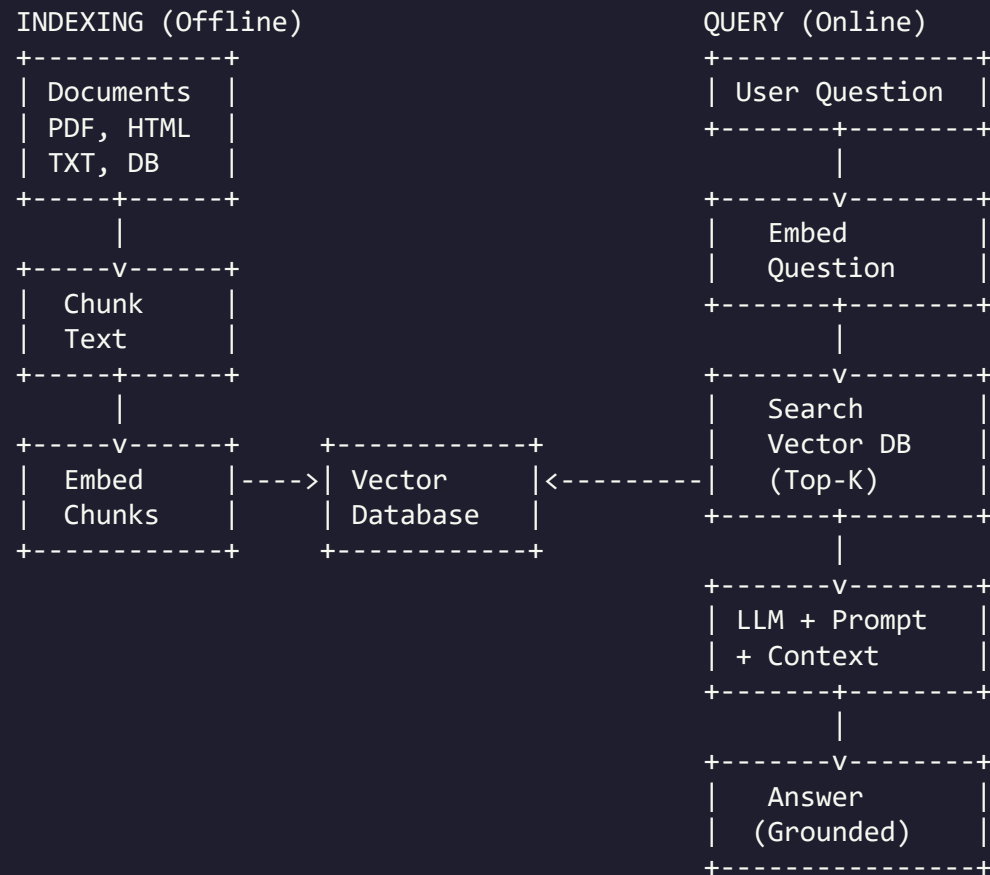
- **1. Ingest:** Load raw documents (PDFs, web pages, text files, databases)
- **2. Chunk:** Split documents into smaller pieces
- **3. Embed:** Convert each chunk into a numerical vector using an embedding model
- **4. Store:** Save vectors + metadata in a vector database

Query Phase (online, per user question):

- **5. Retrieve:** Embed the user's question, search the vector DB for similar chunks
- **6. Generate:** Pass retrieved chunks + question to the LLM, get a grounded answer

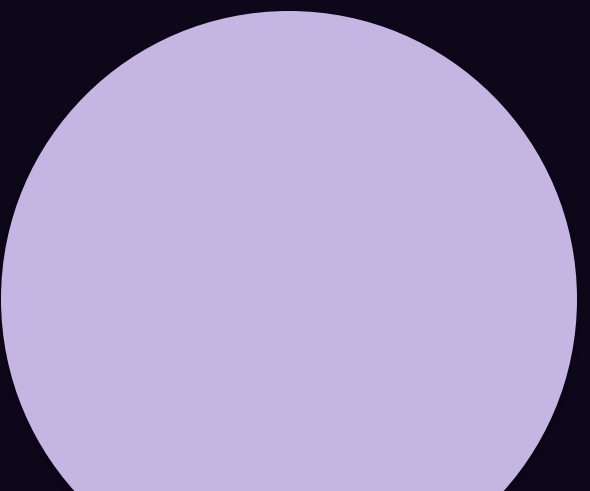
This is the architecture. Every variation of RAG is a modification of one or more of these steps.

RAG Pipeline - Visual



Part 3: Deep Dive - Chunking

How you split your documents determines how good your retrieval will be. This is one of the most impactful decisions in a RAG system.



Why Chunking Matters

You can't embed an entire 50-page PDF as a single vector. The embedding would be too diluted to match specific questions. And you can't embed individual words, because there would be too little context. Chunking is the art of splitting documents into pieces that are small enough to be specific but large enough to be meaningful.

Bad chunking = bad retrieval = bad answers, no matter how good your LLM is.

The chunk is the atomic unit of your RAG system. If the right information is split across two chunks, or buried inside a chunk that's too large, retrieval will fail silently. The LLM will never see what it needs.

Chunk Size and Overlap - The Key Parameters

Chunk size is a tradeoff. Smaller chunks (200-500 tokens) give more precise retrieval. The retrieved chunk is more likely to be exactly about the question. But they lose context: the model sees a fragment, not the full picture. Larger chunks (800-1500 tokens) preserve more context but may dilute the embedding. A chunk about three different topics won't match any of them well.

Overlap ensures that information at chunk boundaries isn't lost. If you split without overlap and a key sentence falls on the cut, it gets torn in half. A 10-20% overlap (for example, 100 tokens overlap on a 500-token chunk) is standard.

There is no universal best chunk size. It depends on your documents, your questions, and your embedding model's sweet spot. This is why evaluation matters, and we'll get there later today.

Metadata - The Secret Weapon of Good RAG

- **What is metadata?** Extra information you attach to each chunk beyond the text itself. Stored alongside the vector in the database.
- **Common metadata fields:** Source filename, page number, section/heading, document date, author, document type, URL.
- **Why it matters for retrieval:** Metadata enables filtered search. "Find chunks about revenue from the 2024 annual report" uses a filter to narrow the search space before vector similarity kicks in.
- **Why it matters for generation:** You can include metadata in the prompt: "The following information comes from [source, page X]..." This gives the LLM provenance, and the user can verify.
- **Why it matters for evaluation:** When something goes wrong, metadata tells you which document and which chunk was retrieved. Essential for debugging.

Chunking Strategies

- **Fixed-size chunking:** Split every N characters/tokens with overlap. Simple, fast, works surprisingly well. Typical: 500-1000 tokens with 50-200 token overlap. Overlap prevents information from being cut at boundaries.
- **Recursive character splitting:** LangChain's default. Tries to split on paragraphs first, then sentences, then words, then characters. Respects natural text boundaries. Best general-purpose method.
- **Sentence-based splitting:** Split on sentence boundaries. Good for precise retrieval. Each chunk = 3-5 sentences.
- **Semantic chunking:** Use an embedding model to detect topic shifts. Split when the semantic similarity between consecutive sentences drops below a threshold. Most intelligent, but slowest.
- **Document-structure-aware:** Use headings, sections, markdown structure, HTML tags. If your docs have clear structure, exploit it.

Chunking in Practice

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

text = """
Artificial intelligence (AI) is intelligence demonstrated by machines,
as opposed to natural intelligence displayed by animals and humans.
AI research has been defined as the field of study of intelligent agents,
which refers to any system that perceives its environment and takes actions
that maximize its chance of achieving its goals.

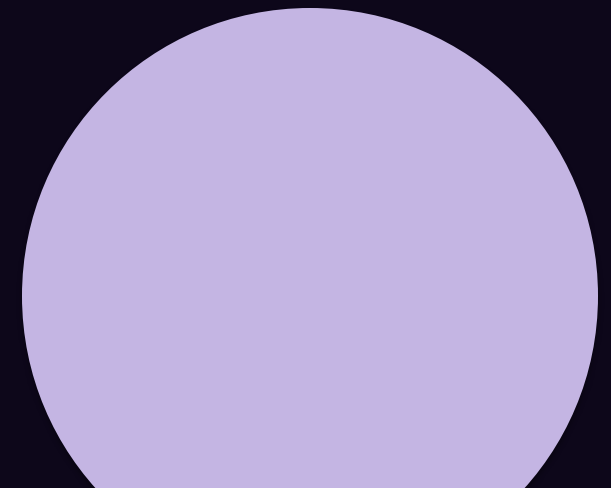
The term 'artificial intelligence' had previously been used to describe
machines that mimic cognitive skills associated with the human mind,
such as 'learning' and 'problem-solving'.

Machine learning is a subset of AI that focuses on building systems that
learn from data. Deep learning uses neural networks with multiple layers.
Transformer models, introduced in 2017, revolutionized NLP.
"""

for size in [100, 200, 500]:
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=size, chunk_overlap=int(size * 0.2), length_function=len
    )
    chunks = splitter.split_text(text)
    print(f"Chunk size: {size}, Overlap: {int(size*0.2)}, Chunks: {len(chunks)}")
    for i, c in enumerate(chunks):
        print(f"  Chunk {i} ({len(c)} chars): {c[:80]}..." )
```


Part 4: Deep Dive - Embeddings and Embedding Models

Embeddings are the bridge between human language and machine-searchable vectors. Your choice of embedding model directly impacts retrieval quality.



Embeddings Refresher

An embedding model takes a piece of text and outputs a dense vector of floating-point numbers (typically 384 to 3072 dimensions). Texts with similar meaning end up close together in vector space, even if they use different words.

"The cat sat on the mat" and "A cat rested on the rug" would have very similar vectors. "Stock market crashes" and "banana bread recipe" would be far apart.

The key insight for RAG: we embed both the document chunks and the user's question using the same model, then find chunks whose vectors are closest to the question's vector. This is semantic search: matching by meaning, not keywords.

Embedding Models - How They Differ

- **Dimensions:** The size of the output vector. More dimensions can capture more nuance but cost more storage and compute. Ranges from 384 (small models) to 3072 (OpenAI large).
- **Max input tokens:** How much text the model can embed at once. Typically 512 tokens (older models) to 8192 tokens (newer models). This interacts with your chunk size. If your chunk is longer than the model's limit, it gets truncated silently.
- **Training data and domain:** Models trained on general web data vs. scientific papers vs. code. Domain match matters.
- **Multilingual support:** Some models handle English only; others handle 100+ languages. Critical if your documents aren't English-only.
- **Speed and cost:** Local models are free but slower. API models are fast but cost per token.
- **Quality (benchmarks):** MTEB leaderboard (Massive Text Embedding Benchmark) ranks models on retrieval, classification, clustering, and more.

Popular Embedding Models - Comparison

Model	Provider	Dims	Max Tokens	Multilingual	Cost	Notes
all-MiniLM-L6-v2	HuggingFace (local)	384	256	Limited	Free	Fast, good starter, small dim
nomic-embed-text	Ollama (local)	768	8192	Yes	Free	Great local option, long context
mxbai-embed-large	Ollama (local)	1024	512	Yes	Free	Higher quality, more dims
text-embedding-3-small	OpenAI (API)	1536	8191	Yes	\$0.02/1M tok	Good cost/quality balance
text-embedding-3-large	OpenAI (API)	3072	8191	Yes	\$0.13/1M tok	Highest quality from OpenAI
voyage-3	Voyage AI (API)	1024	32000	Yes	\$0.06/1M tok	Top MTEB, very long context
text-embedding-004	Google (API)	768	2048	Yes	Free tier	Free via Google AI Studio

How Similarity Search Works - Cosine Similarity

When we search for relevant chunks, we compute the cosine similarity between the question vector and every chunk vector in the database. Cosine similarity measures the angle between two vectors, ignoring magnitude. It is purely about direction.

Score ranges from -1 (opposite) to 1 (identical). In practice, relevant results score 0.7-0.95, and irrelevant ones score below 0.5.

Other distance metrics exist (Euclidean distance, dot product), but cosine similarity is the default for text embeddings and works best in most cases.

Vector databases handle this computation efficiently. You don't loop through every vector yourself; they use approximate nearest neighbor algorithms (HNSW, IVF) to search millions of vectors in milliseconds.

Mini-Exercise 2: Embeddings and Similarity

```
from langchain_ollama import OllamaEmbeddings
import numpy as np

embed_model = OllamaEmbeddings(model="nomic-embed-text")

sentences = [
    "The cat sat on the mat",
    "A feline rested on the rug",
    "Dogs are loyal companions",
    "The stock market crashed yesterday",
    "Python is a programming language",
]

vectors = embed_model.embed_documents(sentences)

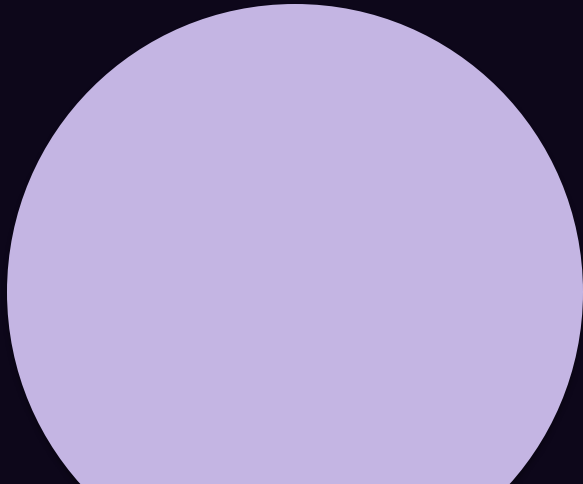
query = "Where is the cat?"
query_vec = embed_model.embed_query(query)

def cosine_sim(a, b):
    a, b = np.array(a), np.array(b)
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

print(f"Query: '{query}'\n")
for sent, vec in zip(sentences, vectors):
    sim = cosine_sim(query_vec, vec)
    print(f" {sim:.4f} <-- '{sent}'")
```

Part 5: Deep Dive - Vector Databases

Where your embeddings live, and how you search them at scale.



Vector Databases - What and Why

A vector database is a specialized database optimized for storing, indexing, and searching high-dimensional vectors. Unlike a traditional database where you query by exact match (SQL WHERE), a vector DB queries by similarity: "find me the 5 vectors most similar to this query vector."

Why not just use a Python list? It works for 100 chunks, but not for 1 million. Vector DBs use specialized indexing algorithms (HNSW, IVF-Flat, Product Quantization) that enable approximate nearest neighbor search in milliseconds across millions of vectors.

They also handle metadata filtering, persistence, concurrent access, and scaling.

Vector Databases - Comparison

Database	Type	Persistence	Scaling	Metadata Filter	Best For
ChromaDB	Embedded (in-process)	Local files	Single machine	Yes	Prototyping, small projects, learning. Zero setup.
FAISS	Library (Meta)	In-memory (manual save)	Single machine	Manual	Research, raw speed. Not a full DB.
Qdrant	Client-server	Disk + memory	Horizontal	Rich filtering	Production. Docker or cloud. Rust-based, fast.
Pinecone	Fully managed cloud	Cloud	Fully managed	Yes	Production without ops. Serverless. Pay-per-use.
Weaviate	Client-server	Disk	Horizontal	GraphQL-style	Production. Built-in vectorization. Hybrid search.
pgvector	PostgreSQL extension	Disk (Postgres)	Via Postgres	Full SQL	Teams already using PostgreSQL.

ChromaDB - Quick Start

```
import chromadb

# Create a client (in-memory for learning, persistent for real use)
client = chromadb.Client() # In-memory
# client = chromadb.PersistentClient(path="./db") # Persistent

# Create a collection (like a table)
collection = client.create_collection(
    name="my_documents",
    metadata={"hnsw:space": "cosine"} # Use cosine similarity
)

# Add documents with metadata
collection.add(
    documents=["Doc text 1", "Doc text 2", "Doc text 3"],
    metadatas=[{"source": "file1.pdf"}, {"source": "file2.pdf"}, {"source": "file1.pdf"}],
    ids=["chunk_1", "chunk_2", "chunk_3"]
)

# Query - find 3 most similar chunks
results = collection.query(
    query_texts=["my question"],
    n_results=3
)
print(results["documents"]) # Top 3 most similar chunks
print(results["metadatas"]) # Their metadata
print(results["distances"]) # Similarity distances
```

Mini-Exercise 3: Build a Tiny Vector Store

```
import chromadb
from langchain_ollama import OllamaEmbeddings

facts = [
    "Lebanon is a country in the Middle East on the Mediterranean Sea.",
    "Beirut is the capital and largest city of Lebanon.",
    "The cedars of Lebanon are famous and appear on the Lebanese flag.",
    "Python was created by Guido van Rossum and first released in 1991.",
    "PyTorch is a deep learning framework developed by Meta's AI lab.",
    "Transformers were introduced in the Attention Is All You Need paper in 2017.",
]
sources = ["geo", "geo", "geo", "prog", "prog", "AI"]

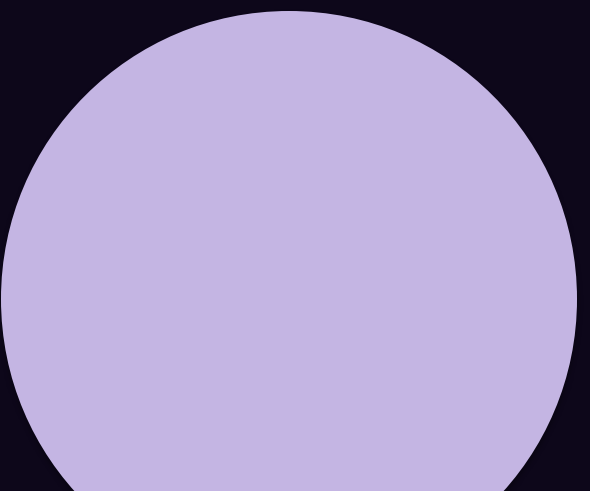
embed_model = OllamaEmbeddings(model="nomic-embed-text")
vectors = embed_model.embed_documents(facts)

client = chromadb.Client()
col = client.create_collection("facts", metadata={"hnsw:space": "cosine"})
col.add(documents=facts, embeddings=vectors,
        metadatas=[{"topic": s} for s in sources],
        ids=[f"fact_{i}" for i in range(len(facts))])

for q in ["What is the capital of Lebanon?", "Who made PyTorch?", "Tell me about trees"]:
    q_vec = embed_model.embed_query(q)
    res = col.query(query_embeddings=[q_vec], n_results=2)
    print(f"\nQuery: '{q}'")
    for doc, meta, dist in zip(res['documents'][0], res['metadatas'][0], res['distances'][0]):
        print(f"  [{dist:.4f}] [{meta['topic']}] {doc[:80]}")
```

Part 6: Retrieval and Generation

We have chunks in a vector DB. Now let's build the query pipeline that retrieves context and generates grounded answers.



Retrieval - Finding the Right Chunks

Retrieval is the most critical step. If you retrieve the wrong chunks, the LLM will either hallucinate (ignoring irrelevant context) or give a wrong answer (misled by wrong context).

The retrieval step works as follows: embed the user's question using the same embedding model used during indexing, search the vector database for the top-K most similar chunks (typically K=3 to 10), and optionally filter by metadata before or during search.

A critical pitfall: the question and the answer are often phrased very differently. The user asks "How much revenue did we make?" but the relevant chunk says "Total fiscal year earnings amounted to \$4.2M." The embedding model needs to understand that these are semantically related. This is why embedding model quality matters so much, and why advanced techniques like query rewriting help.

Generation - The RAG Prompt

Once you have the retrieved chunks, you construct a prompt that gives the LLM both the question and the context:

```
rag_prompt = """You are a helpful assistant. Answer the user's question  
based ONLY on the following context. If the context doesn't contain  
the answer, say 'I don't have enough information to answer that.'
```

```
Context:  
{context}
```

```
Question: {question}
```

```
Answer: """
```

Key prompt engineering decisions for RAG:

- Instruct the model to only use the provided context (reduces hallucination)
- Tell the model what to do when context is insufficient
- Optionally ask the model to cite which chunk(s) it used
- Keep the context focused: more chunks isn't always better (noise dilutes signal)

Complete RAG Pipeline - Query Phase

```
# --- QUERY PHASE ---

question = "What is the main topic of the document?"

# 5. Retrieve - Embed question and search
q_vec = embed_model.embed_query(question)
results = collection.query(query_embeddings=[q_vec], n_results=3)

# Build context from retrieved chunks
context = "\n\n---\n\n".join(results["documents"][0])

print("Retrieved chunks:")
for i, (doc, dist) in enumerate(zip(results["documents"][0], results["distances"][0])):
    print(f"  Chunk {i+1} (distance: {dist:.4f}): {doc[:100]}...")

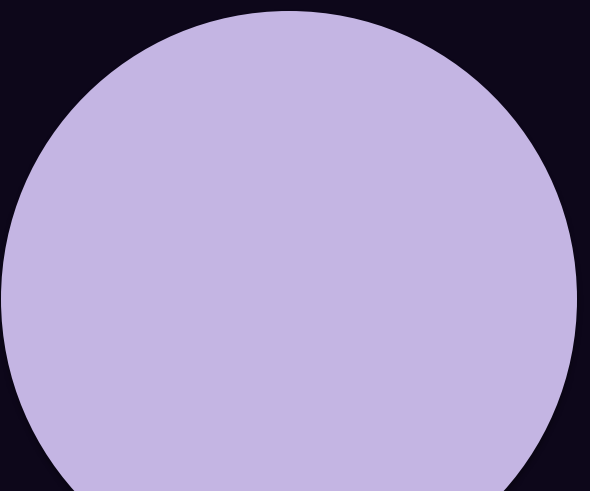
# 6. Generate - Pass context + question to LLM
llm = ChatOllama(model="llama3", temperature=0)

messages = [
    SystemMessage(content=(
        "Answer the question based ONLY on the provided context. "
        "If the context doesn't contain the answer, say so. "
        "Be concise and accurate. Cite which part of the context you used."
    )),
    HumanMessage(content=f"Context:\n{context}\n\nQuestion: {question}")
]

response = llm.invoke(messages)
print(f"\nAnswer: {response.content}")
```

Part 7: Simple vs. Better RAG

The basic pipeline works, but production RAG requires improvements at every step.



Where Simple RAG Fails

- **Wrong chunks retrieved:** The question and the relevant content use different vocabulary. "What are the side effects?" vs. a chunk that says "Adverse reactions include..."
- **Chunk too large / too small:** Key info diluted in a big chunk, or split across two small chunks that aren't both retrieved.
- **Top-K too low:** The answer is in chunk #6, but you only retrieve top-3.
- **Top-K too high:** You retrieve 10 chunks, 7 are irrelevant, and the LLM gets confused by the noise.
- **Metadata not leveraged:** User asks about Q3 2024, but you retrieve Q1 2023 data because the vectors are similar.
- **Multi-hop questions:** "Compare the revenue in Q3 and Q4" requires info from two different sections. Single retrieval may only find one.

Better RAG: Re-Ranking

The idea: retrieve more chunks than you need (e.g., top-20), then use a second, more powerful model to re-score and re-order them. Keep only the best 3-5 for the LLM.

Why? Vector similarity search is fast but approximate. A re-ranker (cross-encoder) is slower but much more accurate because it sees the query and document together, not as separate embeddings. This is a two-stage pipeline: fast recall (vector search) then precise re-ranking.

```
# Conceptual example
initial_results = collection.query(query_embeddings=[q_vec], n_results=20)

# Re-rank with a cross-encoder
from sentence_transformers import CrossEncoder
reranker = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2")

pairs = [(question, doc) for doc in initial_results["documents"][0]]
scores = reranker.predict(pairs)

# Sort by re-ranker score, keep top 3
top_idx = sorted(range(len(scores)), key=lambda i: scores[i], reverse=True)[:3]
best_chunks = [initial_results["documents"][0][i] for i in top_idx]
```

Popular re-rankers: Cohere Rerank API, cross-encoder/ms-marco-MiniLM-L-6-v2 from HuggingFace, Jina Reranker.

Better RAG: Multi-Query Retrieval

The problem: the user's question might not match the vocabulary of the relevant chunks. "How do I fix a slow computer?" might miss chunks about "performance optimization" or "memory management."

The solution: use an LLM to generate 3-5 alternative phrasings, run vector search for each variant, and combine results.

```
# Generate query variants using the LLM
messages = [
    SystemMessage(content=(
        "Generate 3 different versions of the following question "
        "to help find relevant documents. Return only the questions, one per line."
    )),
    HumanMessage(content=f"Original question: {question}")
]

variants_response = llm.invoke(messages)
variant_queries = variants_response.content.strip().split("\n")

# Search with each variant and combine results
all_results = set()
for vq in [question] + variant_queries:
    vq_vec = embed_model.embed_query(vq)
    res = collection.query(query_embeddings=[vq_vec], n_results=3)
    for doc in res["documents"][0]:
        all_results.add(doc)

# Now you have broader, more diverse retrieval
```

Better RAG: Additional Techniques

- **Hybrid search:** Combine vector search (semantic) with keyword search (BM25/TF-IDF). Catches things vector search misses like exact names, codes, and IDs. Weaviate and Qdrant support this natively.
- **Contextual compression:** After retrieval, use an LLM to extract only the relevant sentences from each chunk. Reduces noise in the context window.
- **Parent-child chunking:** Index small chunks for precise retrieval, but return the larger parent chunk to the LLM for context. Best of both worlds.
- **Query routing:** Different questions go to different indexes or retrieval strategies. "What is X?" goes to vector search. "Show me all Y from 2024" goes to metadata filter.
- **Self-RAG:** The LLM decides whether it needs retrieval, retrieves if needed, then critiques its own retrieved context before generating.

RAG Improvement Spectrum

Simple RAG

- Fixed chunking
- Single embedding model
- Top-3 retrieval
- Basic prompt
- No evaluation



Production RAG

- Optimized chunk size
- Metadata filtering
- Re-ranking
- Good prompt engineering
- Human evaluation



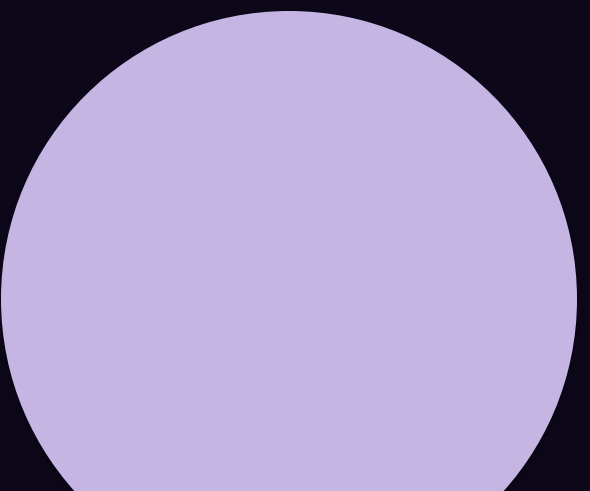
Advanced RAG

- Multi-query
- Hybrid search
- Parent-child chunks
- Query routing
- Self-RAG
- Auto evaluation

Start simple. Measure. Improve what's broken. Don't over-engineer on day one.

Part 8: Basic Evaluation

If you can't measure it, you can't improve it. How to know if your RAG system is actually working.



Evaluating RAG - What to Measure

- **Retrieval quality:** Did you retrieve the right chunks? Measured by: precision (what fraction of retrieved chunks are relevant?), recall (did you retrieve all the relevant chunks?), MRR (Mean Reciprocal Rank: how high is the first relevant result?).
- **Generation quality:** Is the answer correct, complete, and grounded in the context? Harder to measure automatically.
- **Faithfulness:** Does the answer only use information from the retrieved context? Or does the LLM hallucinate beyond it?
- **Answer relevance:** Does the answer actually address the user's question?

Basic Evaluation: The Human Approach

Human evaluation is the most practical approach. Build a small test set of 10-20 question-answer pairs where you know the correct answer and which document(s) it comes from.

For each test question, run your RAG pipeline and check:

- **Retrieval check:** Are the retrieved chunks the ones that contain the answer? (Yes / No per chunk)
- **Answer correctness:** Is the generated answer factually correct? (1-5 scale, or correct / partial / wrong)
- **Faithfulness:** Did the LLM stick to the context or add stuff? (faithful / hallucinated)
- **Source attribution:** If you asked the model to cite sources, are the citations correct?

Log results in a spreadsheet. This gives you a baseline. Change one thing (chunk size, embedding model, prompt, top-K), re-run, and compare. This is the evaluation loop that drives RAG improvement.

Automated RAG Evaluation Tools

- **RAGAS:** Open-source framework that uses LLMs to evaluate RAG. Metrics: faithfulness, answer relevance, context precision, context recall. Requires ground truth for some metrics.
- **LangSmith:** LangChain's observability platform. Traces every step of your chain: what was retrieved, what was sent to the LLM, what was generated. Essential for debugging.
- **Phoenix (Arize):** Open-source tool for tracing and evaluating LLM apps. Visualize embeddings, trace retrievals.

THANK YOU

Questions?

ADDRESS

Mkalles, main road, bld.
757, Lebanon

CONTACT

Phone +961 70 478 758
Email inmind.academy@inmind.ai
Website www.inmind.academy

SOCIAL MEDIA

